

Telegram autotests — setup and run

This section describes how to run the `test_*_telegram_mock.py` suite against a local OmegaClaw container that talks to the real `api.telegram.org` over Telegram's bot-to-bot mode (Bot API 10.0, May 2026). A second "driver" bot plays the test user, sending prompts to the agent bot and reading its replies. The LLM is still mocked (`provider="Test"`, deterministic answers from `Autotests/mock/llm.py`); only the message-delivery transport differs from `Autotests/mock/`.

The 26 tests in this directory mirror `Autotests/mock/test*_mock.py` 1:1 — same mock-LLM answers, same prompts, same assertions — and are listed at the end of this document.

1. Prerequisites

- Docker engine on the host.
- Repository checked out, working from its root.
- Python virtual environment under `Autotests/venv` with `pytest` installed.
- Two Telegram bots, both opted into bot-to-bot mode in BotFather:
 - the **agent bot** — receives prompts, runs OmegaClaw inside the container;
 - the **driver bot** — sends prompts to the agent and collects its replies on behalf of `pytest`.
- An open private chat between the two bots (initiated once via BotFather or by sending `/start` from each side).

2. Configure BotFather

For each of the two bots, in BotFather:

1. `/mybots` → select the bot → **Bot Settings** → enable bot-to-bot mode (Bot API 10.0, May 2026), so the bot can send to and receive messages from other bots.
2. Record the bot token (`TG_BOT_TOKEN` for the agent bot, `TG_DRIVER_TOKEN` for the driver bot).

3. Build the local image

The mock LLM infrastructure is part of the source tree, so the image must be built locally rather than pulled from the registry.

```
docker build -t omegaclaw:mock .
```

4. Start the container with the Test provider and the Telegram channel

The container connects back to the host on TCP port 9765 to reach the mock LLM controller (`0.0.0.0` on the host, `172.17.0.1` from inside the default docker bridge). The Telegram adapter inside the container talks directly to `api.telegram.org` — there is no

TG_API_BASE override.

```
docker run -d -it \
  --name omegaclaw_tg \
  --user 65534:65534 \
  --init \
  --network bridge \
  --security-opt no-new-privileges:true \
  --volume omegaclaw-tg-memory:/PeTTa/repos/OmegaClaw-Core/memory/ \
  --tmpfs /tmp:size=64m,mode=1777,exec \
  --tmpfs /run:size=16m,mode=755 \
  --tmpfs /var/tmp:size=64m,mode=1777,exec \
  -e TEST_API_KEY=172.17.0.1 \
  -e OMEGACLAW_AUTH_SECRET=0000 \
  omegaclaw:mock \
  commchannel="telegram" \
  TG_BOT_TOKEN="<agent_bot_token>" \
  TG_POLL_TIMEOUT="5" \
  provider="Test" \
  embeddingprovider="Local"
```

Notes:

- commchannel="telegram" selects the Telegram adapter inside src/channels.metta.
- TG_BOT_TOKEN is the **agent bot** token (the bot that the OmegaClaw loop runs as).
- provider="Test" selects the mock LLM dispatcher.
- embeddingprovider="Local" keeps the embedding model in-process (no network call).
- TEST_API_KEY=172.17.0.1 is the host's docker-bridge address used by the mock LLM provider.
- OMEGACLAW_AUTH_SECRET=0000 matches the value the autouse _tg_authenticate fixture sends as auth 0000 once per session.

Wait until the container has started the Telegram poll loop:

```
docker logs omegaclaw_tg 2>&1 | grep -E "Telegram"
```

5. Configure the test environment

Export the variables the test harness reads.

```
export OMEGACLAW_CONTAINER=omegacrawl_tg
export TG_BOT_TOKEN="<agent_bot_token>"
export TG_DRIVER_TOKEN="<driver_bot_token>"
export TG_MIRROR_CHAT_ID="<optional_chat_id_for_mirror>"
export OMEGACLAW_GIT_TOKEN=<github_pat> # only required by test_git_p
```

Variable	Required	Description
OMEGACLAW_CONTAINER	Yes	Name passed to docker exec from the harr Must equal --name above.
TG_DRIVER_TOKEN	Yes	Driver bot token. Tests are skipped if unset.

TG_BOT_TOKEN	Yes (or TG_AGENT_USERNAME)	Agent bot token. Used by the harness to auto-derive the agent's @username via Telegram's getMe endpoint.
TG_AGENT_USERNAME	No	Explicit override for the agent bot's @username (without @). If set, the getMe probe is skipped. Useful in air-gapped CI where the host cannot reach api.telegram.org.
TG_MIRROR_CHAT_ID	No	Chat id (numeric, can be negative for groups) that receives a mirror of the bot-to-bot conversation and per-test PASS/FAIL/SKIP lines.
OMEGACLAW_GIT_TOKEN	No	GitHub PAT used by test_git_push_to_remote_telegram_repo. The test is skipped if this variable is unset.

6. Run the suite

```
cd Autotests
source venv/bin/activate
pytest -s -v mock_telegram/test*_telegram_mock.py
```

The LLM mock controller and the `RealTgDriver` are provided by session-scoped fixtures in `mock_telegram/conftest.py`, so both are started once per pytest session. Expected output: 26 passed (plus 1 skipped if `OMEGACLAW_GIT_TOKEN` is not set).

7. Tear down

```
docker rm -f omegaclaw_tg
docker volume rm omegaclaw-tg-memory
```

Tests description

All 26 tests are 1:1 mirrors of the corresponding `Autotests/mock/test*_mock.py` files. The mock-LLM answer, prompt body, prepared fixtures, and assertions are identical to the IRC variants; the only difference is the message-delivery transport. Where the IRC variant calls `helpers.send_prompt(prompt)`, the Telegram variant calls `tg_send_prompt(tg, prompt)`, which makes the driver bot send `sendMessage(@agent, prompt)` to the agent bot via `api.telegram.org`. Because the LLM is deterministic, no `try_with_clarification` retries are needed — every test either passes on the first attempt or fails outright.

Creating files

1. test_create_file_telegram_mock.py

Creates `/tmp/testcat/hello.txt` containing exactly `Hello`.

- **Mock answer:** `(shell "mkdir -p /tmp/testcat") (write-file "/tmp/testcat/hello.txt" "Hello")`.
- **Checks:** directory exists, file exists, permissions start with `-rw`, `mtime ≥ test start`, content is `Hello` (with or without trailing newline).

2. test_create_empty_file_telegram_mock.py

Creates an empty `/tmp/test_empty/hello.txt`.

- Mock answer: `(shell "mkdir -p /tmp/test_empty") (write-file "/tmp/test_empty/hello.txt" "")`.
- Checks: file is created, `cat` returns an empty string.

3. test_create_script_telegram_mock.py

Creates `/tmp/test_script/date.sh` — a shell script that prints the current date.

- Mock answer: `(shell "mkdir -p /tmp/test_script") (write-file "/tmp/test_script/date.sh" "#!/bin/bash\ndate\n") (shell "chmod +x /tmp/test_script/date.sh")`.
- Checks: file exists, is executable (`x` in permissions), harness runs it via `sh date.sh` and verifies the output contains the current year (UTC or local).

Editing files

4. test_edit_add_timestamp_telegram_mock.py

Appends the current timestamp as a new line to a pre-created `note.txt`.

- Mock answer: `(shell "date -Iseconds >> {TARGET_FILE}")`.
- Checks: mtime changed, last line of the file contains ≥ 4 digits.

5. test_edit_delete_line_telegram_mock.py

Deletes the second line from a pre-created 3-line file.

- Mock answer: `(shell "sed -i 2d {TARGET_FILE}")`.
- Checks: mtime changed, exactly 2 lines remain, lines 1 and 3 intact.

6. test_edit_append_line_telegram_mock.py

Appends a 4th line `Delta` to a pre-created 3-line file (`Alpha / Bravo / Charlie`).

- Mock answer: `(shell "printf '%s\n' Delta >> {TARGET_FILE}")`.
- Checks: mtime changed, 4 lines total, first 3 unchanged, 4th line equals `Delta`.

7. test_convert_format_telegram_mock.py

Converts `document.md` $\rightarrow$ `document.txt`, preserving textual content.

- Mock answer: `(shell "cp {SOURCE_FILE} {DEST_FILE}")`.
- Checks: `.txt` exists, contains the keywords `My Title`, `Some paragraph text`, `item one`, `item two`.

Running shell scripts

8. test_run_error_script_telegram_mock.py

Runs a syntactically broken pre-created script and captures stdout + stderr to a file.

- Mock answer: `(shell "sh {SCRIPT_FILE} > {OUTPUT_FILE} 2>&1")`.
- Checks: `output.txt` exists, contains the literal `start` (stdout) AND at least one of `error` / `syntax` / `unexpected` / `missing` / `not found` (stderr); container is still alive.

9. test_run_repeated_telegram_mock.py

Runs `dateupdate.sh` exactly 10 times in a row.

- Mock answer: ten consecutive `(shell "{SCRIPT_FILE}")` calls (one per run).
- Checks: `update.txt` exists with `mtime ≥ start`, has $≥ 10$ lines, every line contains date-like digits.

Internet search

10. test_search_basic_telegram_mock.py

Sends prompt "What is SingularityNet? Search the web."

- Mock answer: `(send "SingularityNet (SNET) is a decentralized AI marketplace founded by Ben Goertzel. Its native token AGIX powers a network of AI agents and services, and it is part of the broader ASI Alliance ecosystem.")`.
- Checks: the reply contains at least one of `singularitynet`, `agi`, `blockchain`, `decentralized`, `marketplace`, `goertzel`.

11. test_search_weather_telegram_mock.py

Returns a synthetic Valencia weather reply (the live variant cross-checks against open-meteo; the mock variant fixes a synthetic reference `REF_TEMP_C = 18.0` and stays fully offline).

- Mock answer: `(send "Current weather in Valencia, Spain: about 18.0°C.")`.
- Checks: the reply contains a plausible Celsius number (range `[-20; 50]`); at least one of those numbers is within $±10$ °C of the synthetic reference `18.0 °C`.

12. test_search_invalid_telegram_mock.py

Asks about a gibberish string.

- Mock answer: `(send "No results found for <gibberish>. The string appears to be gibberish – no meaningful matches.")`.
- Checks: the reply contains a negation phrase (`no results`, `not found`, `gibberish`, `nonsense`, `no meaning`, `unknown`, ...).

13. test_tavily_search_telegram_mock.py

Live variant exercises the external Tavily uAgent. The mock variant cannot reach it deterministically, so the mocked response delivers the answer directly via `(send ...)` and the assertion narrows to "did the agent surface a real Fetch.ai-specific reply".

- **Mock answer:** (send "Fetch.ai (FET) is a decentralized AI blockchain platform powering autonomous economic agents (uAgents). Recent news covers the ASI Alliance roadmap, FET token activity, and integration work with SingularityNET and CUDOS."). The tavily-search skill itself is **not** invoked under the mock.
- **Checks:** a (send ...) exists whose body contains at least one strict Fetch keyword (fetch.ai, fetch ai, fet , asi alliance, humayun, uagent, decentralized, blockchain, token) AND none of the delivery-error markers (delivery failed, tavily-search failed, currently unavailable, ...).

14. test_technical_analysis_telegram_mock.py

Live variant exercises the external technical-analysis uAgent. The mock variant cannot reach it deterministically, so the mocked response delivers the TA summary directly via (send ...) and the assertion narrows to "did the agent surface TA-style content for the requested ticker".

- **Mock answer:** (send "AAPL (Apple) is showing bullish momentum: RSI is rising, MACD crossed above its signal line, and the 50-day SMA is above the 200-day. Composite indicators point to a buy signal with strong trend strength."). The technical-analysis skill itself is **not** invoked under the mock.
- **Checks:** a (send ...) exists whose body mentions the ticker (aapl or apple) AND at least one TA indicator (rsi, macd, sma, bullish, bearish, buy signal, trend, momentum, ...) AND none of the delivery-error markers.

Memory

15. test_memory_chromadb_telegram_mock.py

Requests the agent to remember a fact tagged with marker CI-SMOKE-<run_id>.

- **Mock answer:** (remember "Unique smoke marker CI-SMOKE-<run_id> was emitted by CI.").
- **Checks:** (remember ...) was invoked with the marker; vector count in the embeddings table of chroma.sqlite3 grew by ≥ 1 .

16. test_memory_history_telegram_mock.py

Sends "Acknowledge with one short line that you received marker <run_id>." and verifies the entry in history.metta.

- **Mock answer:** (send "Acknowledged marker <run_id>").
- **Checks:** an s-exp record referencing REQ-<run_id> appears in history; the agent issued (send ...); file mtime and size grew.

17. test_skill_metta_telegram_mock.py

Asks the agent to evaluate a short MeTTa expression and report the result.

- **Mock answer:** (metta "(+ 2 2)") (send "The metta skill evaluated (+ 2 2) and returned 4.").

- Checks: `(metta ...)` was invoked; the agent then issued a `(send ...)`. Semantic correctness of the MeTTa expression is not checked — the goal is to exercise the skill.

18. test_skill_pin_telegram_mock.py

Gives a multi-step task ("restarting servers alpha → beta → gamma, just finished alpha") and expects the agent to track progress with `pin`.

- Mock answer: `(pin "Server restart progress: alpha done; beta and gamma pending.") (send "Tracking: alpha done, beta and gamma pending.")`.
- Checks: `(pin ...)` was invoked whose argument references either the `run_id` or one of the keywords `step` / `alpha` / `beta` / `gamma` / `restart` / `server` / `done`; the agent acknowledged via `(send ...)`.

Working with git

19. test_git_pull_public_telegram_mock.py

Agent clones a public repository over anonymous HTTPS, no token.

- Mock answer: `(shell "rm -rf {TARGET_DIR} && git clone {remote} {TARGET_DIR}")`.
- Checks: `.git/` appears, HEAD points to a real commit, ≥ 1 tracked file in HEAD, origin matches the expected remote URL (normalized — trailing / and `.git` ignored).

20. test_git_local_commit_telegram_mock.py

Agent runs `git init`, `git add`, `git commit` locally inside the container.

- Mock answer: chain of `(shell "git -C {TARGET_DIR} init") (shell "...write file...") (shell "git -C {TARGET_DIR} add -A") (shell "git -C {TARGET_DIR} commit -m 'add hello <run_id>')"`.
- Checks: HEAD has at least one commit, commit subject contains the `run_id` (warning, not failure), the file is present in the tree.

21. test_git_push_to_remote_telegram_mock.py

Agent clones a remote, creates branch `qa/run-<id>`, adds a file, commits, and pushes.

- Mock answer: single `(shell "rm -rf ... && git clone ... && cd ... && git checkout -b ... && printf ... > <file> && git add -A && git commit -m '...' && git push -u origin ...")`.
- Parameters via env vars: `OMEGACLAW_GIT_TOKEN` (token; never appears in code) and `OMEGACLAW_GIT_REMOTE` (default `https://github.com/OmegaSing/Test-Repopo`). Test is skipped if the token variable is unset.
- Checks: branch present on remote (GitHub API 200), file present on branch, the shell call included `git push`, credentials wiped on teardown.

Multi-skill tests

22. test_run_create_dirs_telegram_mock.py

Agent writes `mkdirs.sh` and runs it. The script must create `test1`, `test2`, `test3` inside `/tmp/test_dirs/`.

- Mock answer: `(write-file "{SCRIPT_PATH}" "#!/bin/bash\nmkdir -p .../test1 .../test2 .../test3\n") (shell "chmod +x {SCRIPT_PATH}") (shell "{SCRIPT_PATH}")`.
- Checks: all three directories exist with fresh mtimes; agent invoked `(write-file ...)` referencing `mkdirs.sh`; agent invoked `(shell ...)` to run the script. Diagnostics print `wf=<count>`, `sh=<count>`, `perms=<...>` to make stalls obvious.

23. test_memory_episode_telegram_mock.py

Two-turn flow: tells the agent that the user's dog Barney lost a baby tooth, waits 5 seconds, then asks to recall when this happened.

- Turn 1 mock answer: `(remember "Barney the dog lost his first baby tooth at the vet today.")`.
- Turn 2 mock answer: `(query "Barney tooth") (send "Barney the dog lost his first baby tooth on <YYYY-MM-DD>. The milestone is recorded in my notes.")`.
- Checks (turn 1): `(remember ...)` was invoked whose argument contains `tooth` or `Barney`. Checks (turn 2): `(query ...)` or `(episodes ...)` was invoked; the reply contains at least one of `dog` / `tooth` / `lost`; the reply contains the captured seed date in `YYYY-MM-DD` format.

24. test_skill_query_telegram_mock.py

Two-turn flow: plant a unique color (`azure-<run_id>`) via `remember`, wait for embeddings to settle, then ask the agent to recall it via `query` (embedding lookup, not timestamp lookup).

- Turn 1 mock answer: `(remember "My favorite color is azure-<run_id>.") (send "Stored: favorite colour is azure-<run_id>.")`.
- Turn 2 mock answer: `(query "favorite color") (send "Your favorite color is azure-<run_id>.")`.
- Checks (turn 1): `(remember ...)` carried the secret color. Checks (turn 2): `(query ...)` was invoked; the reply mentions the secret color verbatim.

25. test_skill_episodes_telegram_mock.py

Two-turn flow: send a message tagged with a unique keyword (no `remember`), capture the timestamp, then ask the agent to use `episodes` (timestamp lookup, not `query`) to recall what was discussed at that earlier time.

- Turn 1 mock answer: `(send "Acknowledged keyword <marker>.")`.
- Turn 2 mock answer: `(episodes "<seed_ts>") (send "The unique keyword was <marker>.")`.
- Checks (turn 1): the turn is recorded in `history.metta` with a timestamp. Checks (turn 2): `(episodes ...)` was invoked for the seed timestamp; the reply mentions the original marker.

26. test_complex_weather_flow_telegram_mock.py

Four-step pipeline: search NY weather → write `w.txt` with the forecast → write `p.sh` extracting the first Celsius number into `t.txt` → run `p.sh`. Because the mock controls only the LLM dispatch (the network-bound `search` skill is not exercised), the mocked response provides the forecast text directly.

- **Mock answer:** `(write-file "/tmp/wflow/w.txt" "New York tomorrow: clear, high 22 degrees Celsius.") (write-file "/tmp/wflow/p.sh" "#!/bin/bash\ngrep -oE '[0-9]+' /tmp/wflow/w.txt | head -1 > /tmp/wflow/t.txt\n") (shell "chmod +x /tmp/wflow/p.sh") (shell "/tmp/wflow/p.sh")`.
- **Checks:** `w.txt` exists; history contains `(write-file ...)` referencing `w.txt`; `p.sh` exists with executable bit; history contains `(write-file ...)` or `(shell ...)` referencing `p.sh`; `t.txt` exists; history contains `(shell ...)` running `p.sh`; `t.txt` content is a number in the range `[-60; 120]`; content length ≤ 40 characters.